

Diseño y desarrollo de una arquitectura multiagente
basada en LangGraph
para procesamiento de datos y detección de
anomalías industriales

Autor: pendiente

Curso académico: pendiente

Resumen

Este Trabajo Fin de Máster propone una arquitectura multiagente para el procesamiento, modelado y detección de anomalías en entornos industriales. El caso de uso inicial se centra en rodamientos de motores eléctricos monitorizados mediante señales de vibración. La solución combina agentes coordinados mediante LangGraph con ejecutores Python deterministas, de forma que las decisiones se representen mediante contratos estructurados y las transformaciones sobre datos sean reproducibles, audita-
bles y testeables.

Abstract

This Master's Thesis proposes a multi-agent architecture for data processing, modeling and anomaly detection in industrial environments. The initial use case focuses on electric motor bearings monitored through vibration signals. The solution combines agents coordinated with LangGraph and deterministic Python executors, so that decisions are represented through structured contracts and data transformations remain reproducible, auditable and testable.

Índice general

1. Introducción	1
1.1. Contexto	1
1.2. Motivacion	1
1.3. Problema	1
1.4. Objetivo general	2
1.5. Objetivos específicos	2
1.6. Alcance y limitaciones iniciales	2
2. Estado del arte	3
2.1. Mantenimiento predictivo y detección de anomalías	3
2.2. Modelos de detección de anomalías	3
2.3. Sistemas multiagente aplicados a pipelines de datos	3
2.4. Datasets públicos de referencia	4
3. Arquitectura propuesta	5
3.1. Principios de diseño	5
3.2. Separación entre agentes y ejecutores	5
3.3. Nodos previstos	5
3.4. Estado global	6
3.5. Contratos estructurados	7
3.6. Trazabilidad	7
4. Metodología	8
4.1. Caso de uso industrial	8
4.2. Pipeline de datos	8
4.3. Generalización multi-dataset	8
4.4. Manifiesto del dataset	10
4.5. Extracción y perfilado	10
4.6. Estructuración temporal	10
4.7. Particiones	11
4.8. Métricas	11
5. Implementación	12
5.1. Estructura del repositorio	12
5.2. Estado Actual	12
5.3. Estado global implementado	12
5.4. Contratos Pydantic implementados	13
5.5. Artefactos previstos	13
5.6. Adaptadores de Entrada	13

5.7.	Ejecutor de manifiesto	14
5.8.	Ejecutor de perfilado	14
5.9.	Ejecutor de Limpieza	14
5.10.	Ejecutor de Estructuración Temporal	15
5.11.	Ejecutor de Modelado Base	15
5.12.	Ejecutor de Evaluación	15
5.13.	Grafo LangGraph supervisado	16
5.14.	Agente Limpiador LLM	17
5.15.	Agente Estructurador LLM	17
5.16.	Agente Modelador LLM	18
5.17.	Agente Evaluador LLM	19
5.18.	Agente Redactor LLM	20
5.19.	Persistencia Local de Ejecuciones	20
5.20.	Wrapper de Ejecucion Persistida	21
5.21.	Registro Consultable de Ejecuciones	21
5.22.	Protocolo Experimental Local	22
5.23.	API Minima de Consulta	22
5.24.	Contratos Iniciales Multi-Dataset	23
5.25.	Prueba Agentica NASA IMS con Qwen	24
5.26.	Benchmark NASA IMS Sintetico Etiquetado	25
5.27.	Bucle de Reintento Agentic	25
5.28.	Dependencias	26
5.29.	Verificación Actual	26
5.30.	Siguiente Paso de Implementación	27
6.	Experimentos	28
6.1.	Protocolo experimental	28
6.2.	Plan CWRU Isolation Forest Threshold	28
6.3.	Comparacion Agentica de Ventanas CWRU	29
6.4.	Comparacion Agentica de Modelos CWRU	30
6.5.	Prueba Smoke NASA IMS con Agentes Qwen	30
6.6.	Benchmark NASA IMS Sintetico Etiquetado	31
6.7.	Iteracion Agentica sobre NASA IMS Sintetico	31
6.8.	Experimentos posteriores	32
7.	Resultados	33
7.1.	Resultados cuantitativos	33
7.1.1.	Baseline CWRU de Fase 2	33
7.1.2.	Comparacion agentica de ventanas	33
7.1.3.	Comparacion agentica de modelos	34
7.1.4.	Benchmark NASA sintetico y reintentos	34
7.2.	Resultados cualitativos	35
7.3.	Limitaciones observadas	35
8.	Conclusiones	36
8.1.	Conclusiones principales	36
8.2.	Limitaciones	36
8.3.	Trabajo futuro	37

Índice de figuras

Índice de tablas

6.1.	Comparacion local de umbral con Isolation Forest sobre CWRU	29
6.2.	Comparacion agentica de ventanas sobre CWRU	29
6.3.	Comparacion agentica de modelos sobre CWRU	30
6.4.	Run agentica sobre benchmark NASA IMS sintetico etiquetado	31
6.5.	Reintentos agenticos sobre benchmark NASA IMS sintetico	31
7.1.	Comparacion de umbral con Isolation Forest en el split de test CWRU.	33
7.2.	Comparacion agentica de tamanos de ventana en CWRU con 50 % de solape.	34
7.3.	Comparacion agentica de modelos sobre CWRU con ventana de 1024 muestras.	34
7.4.	Reintentos agenticos sobre benchmark NASA sintetico etiquetado.	35

Capítulo 1

Introducción

1.1. Contexto

La detección de anomalías en maquinaria industrial es una tarea relevante para mejorar la fiabilidad, reducir paradas no planificadas y apoyar estrategias de mantenimiento predictivo. En sistemas rotativos, como motores eléctricos, turbinas, bombas o rodamientos, las señales de vibración permiten observar cambios de comportamiento asociados a desgaste, desbalanceo, defectos mecánicos o degradación progresiva.

Este trabajo se centra inicialmente en rodamientos de motores eléctricos, usando como dataset principal CWRU Bearing Dataset [2]. El dataset NASA IMS Bearing se conserva como fuente futura para validar escenarios de degradación run-to-failure [4].

1.2. Motivación

El interés del proyecto no se limita a entrenar un modelo de detección de anomalías. El objetivo es construir una arquitectura capaz de coordinar el flujo completo de datos: ingestión, perfilado, limpieza, estructuración temporal, entrenamiento, evaluación, auditoría e informe técnico. Para evitar transformaciones opacas o no reproducibles, los agentes proponen decisiones estructuradas y los ejecutores deterministas aplican las operaciones reales.

1.3. Problema

El problema abordado es la detección automática de patrones anómalos en señales de vibración procedentes de rodamientos. En el MVP se formulará como una tarea binaria:

- clase normal: operación sin fallo;
- clase anómala: cualquier fallo de rodamiento disponible en el dataset.

La clasificación por tipo de fallo se mantiene como extensión posterior. Esta decisión reduce el alcance inicial y permite validar primero la trazabilidad del pipeline multiagente.

1.4. Objetivo general

Diseñar e implementar una arquitectura multiagente basada en LangGraph para procesar datos industriales de vibración y detectar anomalías de forma reproducible, auditable y extensible.

1.5. Objetivos específicos

1. Definir una estructura de proyecto que separe código, memoria académica y recursos.
2. Seleccionar un caso de uso industrial y datasets públicos adecuados.
3. Diseñar un pipeline de datos reproducible para señales de vibración.
4. Definir contratos estructurados para agentes y ejecutores.
5. Implementar ejecutores deterministas para perfilado, limpieza, estructuración temporal, modelado y evaluación.
6. Construir un MVP local antes de abordar Docker, SLURM o frontend.
7. Generar informes técnicos trazables a partir de métricas y artefactos.

1.6. Alcance y limitaciones iniciales

El alcance inicial queda limitado a ejecución local, ficheros crudos en disco y un dataset principal. No se abordarán en el MVP la ejecución distribuida, streaming industrial, integración SLURM, frontend ni persistencia avanzada. NASA IMS se mantiene como extensión experimental una vez validado el flujo sobre CWRU.

Capítulo 2

Estado del arte

2.1. Mantenimiento predictivo y detección de anomalías

Esta sección desarrollará el contexto de mantenimiento predictivo, monitorización de condición y detección de anomalías en maquinaria rotativa.

Aspectos a cubrir:

- diferencias entre mantenimiento correctivo, preventivo y predictivo;
- papel de las señales de vibración en rodamientos y motores;
- fallos comunes: pista interior, pista exterior, bola y desbalanceo;
- diferencias entre detección de anomalías, diagnóstico y pronóstico.

2.2. Modelos de detección de anomalías

El MVP comparará primero modelos clásicos y reproducibles:

- Isolation Forest;
- One-Class SVM;
- Local Outlier Factor;
- PCA con error de reconstrucción.

Los autoencoders densos y modelos recurrentes se considerarán en fases posteriores, cuando el pipeline de tensores esté validado.

2.3. Sistemas multiagente aplicados a pipelines de datos

Esta sección describirá el uso de arquitecturas multiagente para dividir responsabilidades entre nodos especializados. El enfoque del proyecto separa:

- decisiones de alto nivel tomadas por agentes;
- validación de contratos mediante esquemas estrictos;
- ejecución determinista de transformaciones sobre datos;
- auditoría de rutas, configuraciones, métricas y artefactos.

2.4. Datasets públicos de referencia

El dataset principal será CWRU Bearing Dataset [2, 3, 1]. NASA IMS Bearing Dataset se reserva como extensión para escenarios de degradación temporal [4, 5].

Capítulo 3

Arquitectura propuesta

3.1. Principios de diseño

La arquitectura se basa en cuatro principios:

1. separar código, memoria académica y recursos;
2. mantener los datos crudos inmutables;
3. representar decisiones mediante contratos estructurados;
4. ejecutar transformaciones mediante módulos Python deterministas.

3.2. Separación entre agentes y ejecutores

Los agentes no escriben ni ejecutan código arbitrario sobre los datos. Su papel es proponer configuraciones, interpretar perfiles y decidir la siguiente fase del flujo. Los ejecutores leen artefactos desde disco, aplican operaciones deterministas y devuelven rutas, métricas y logs.

Para que agentes basados en LLM locales puedan actuar como expertos sin depender de conocimiento interno sobre datasets de nicho, la arquitectura introduce el patrón de expediente experto. Antes de cada decisión relevante, un ejecutor determinista resume la evidencia disponible, calcula diagnósticos, enumera opciones soportadas y explicita advertencias o bloqueos. El agente conserva libertad para escoger, ordenar alternativas, justificar su criterio o solicitar una capacidad no implementada, pero su respuesta sigue validándose mediante un contrato Pydantic.

Como extensión futura, se contempla un agente investigador controlado. Su papel no sería ejecutar transformaciones, sino responder solicitudes estructuradas de otros agentes mediante evidencia local o fuentes autorizadas, registrando citas, fecha, confianza y limitaciones. De este modo la arquitectura puede incorporar búsqueda documental o RAG sin conceder navegación libre ni capacidad de ejecutar acciones fuera de los contratos.

3.3. Nodos previstos

El grafo mínimo previsto contiene:

- supervisor;
- agente limpiador;
- ejecutor de limpieza;
- agente estructurador;
- ejecutor de estructuración;
- agente modelador;
- ejecutor de modelado;
- evaluador;
- redactor;
- revisión humana para trabajos costosos.

3.4. Estado global

El estado global de LangGraph no almacenará datasets completos. Debe contener rutas, resúmenes y artefactos:

- identificador de ejecución;
- contexto del proyecto;
- rutas de datos crudos, perfiles, datos limpios, tensores e informes;
- configuraciones de limpieza, estructuración y modelado;
- métricas y evaluación;
- errores, aprobaciones humanas y artefactos generados.

En la implementación inicial se ha definido un modelo canónico ‘TFMStateModel’ validado con Pydantic y una vista ‘TFMState’ basada en ‘TypedDict’ para su uso en LangGraph. Esta separación permite validar contratos de forma estricta antes de entregar el estado al grafo y, al mismo tiempo, mantener una representación ligera y serializable.

El estado inicial del caso CWRU queda fijado en la fase ‘dataset_manifest’, con el nodo siguiente ‘manifest_executor’. El directorio crudo se registra como `codigo/data/raw/cwru_bearing/mat`, el canal principal es ‘DE_time’ y la frecuencia objetivo es 12 kHz. Las rutas de artefactos posteriores permanecen como valores nulos hasta que los ejecutores correspondientes las generen.

3.5. Contratos estructurados

Los contratos iniciales incluyen:

- ‘ProjectContext’, que define dominio, máquina, dataset, objetivo, canal principal y frecuencia objetivo;
- ‘DatasetManifestRow’ y ‘DatasetManifest’, que formalizan el inventario de ficheros crudos y sus etiquetas;
- ‘DatasetProfileSummary’, que resume el perfil sin almacenar señales;
- ‘CleaningConfig’, ‘StructuringConfig’ y ‘ModelingConfig’, que fijan las configuraciones que podrán proponer los agentes;
- ‘SupervisorDecision’, ‘CleaningDecision’, ‘StructuringDecision’, ‘ModelingDecision’, ‘EvaluationDecision’ y ‘ReportDecision’, que estructuran las respuestas de los agentes;
- ‘ExecutorResult’ y resultados especializados, que normalizan las salidas de los ejecutores deterministas;
- ‘MetricsReport’ y ‘EvaluationResult’, que estructuran la evaluación;
- ‘PipelineError’, ‘HumanApproval’ y ‘ArtifactRef’, que mantienen trazabilidad de errores, revisiones y artefactos.

Estos esquemas aplican validación estricta y rechazan campos no declarados. Por ejemplo, una fila normal del manifiesto debe declarar los campos de fallo como nulos, mientras que una fila con fallo debe incluir tipo de fallo y diámetro. De forma análoga, un resultado de ejecutor con estado fallido debe incluir un error estructurado, y una decisión no terminal del supervisor debe indicar el siguiente nodo del grafo.

3.6. Trazabilidad

Cada ejecución debe permitir responder:

- qué datos se usaron;
- qué configuración fue validada;
- que ejecutor produjo cada artefacto;
- qué métricas se obtuvieron;
- qué limitaciones permanecen.

Capítulo 4

Metodología

4.1. Caso de uso industrial

El caso de uso seleccionado es la detección de anomalías en rodamientos de motores eléctricos mediante señales de vibración. La elección permite trabajar con datos industriales públicos, etiquetas interpretables y un flujo temporal adecuado para ventanas, features y tensores.

4.2. Pipeline de datos

El flujo metodológico para el MVP queda definido como:

```
datos .mat crudos
-> manifiesto del dataset
-> extracción de señales
-> normalización de frecuencia de muestreo
-> perfilado estadístico
-> limpieza determinista
-> ventanas temporales
-> features y tensores
-> particiones train/validation/test
-> modelos de anomalías
-> evaluación
-> informe técnico
```

4.3. Generalización multi-dataset

Tras consolidar el MVP sobre CWRU, la siguiente extensión metodológica consiste en separar explícitamente tres niveles: el dataset crudo, el adaptador determinista de dataset y el pipeline común. Esta separación evita que los ejecutores dependan de nombres de fichero o estructuras particulares de un benchmark concreto.

La generalización se plantea mediante un descriptor común de dataset, un manifiesto común con una fila por unidad cruda procesable y un registro de adaptadores. CWRU se mantendrá como benchmark de regresión, mientras que NASA IMS se contempla como primer candidato para validar el sistema con series más largas, múltiples canales y condiciones de degradación menos triviales.

La inspección local de NASA IMS muestra que el paquete se distribuye como un contenedor anidado ('zip', '7z' y 'rar') y que la aplicación deberá distinguir entre detección de formato, selección de adaptador y generación efectiva de manifiesto. En una futura interfaz, el usuario debería ver una previsualización del descriptor del dataset, los formatos detectados, las limitaciones de extracción y si el manifiesto puede generarse directamente o requiere preparar una carpeta preextraída.

Como primer paso ejecutable, el manifiesto NASA IMS se restringe a carpetas ya preextraídas. Cada snapshot temporal se representa como una fila común con etiqueta inicial desconocida, fallo final conocido a nivel de set y metadatos de frecuencia, canales y variante local. Esta decisión permite validar la generalización del pipeline sin incorporar todavía dependencias de extracción ni una política de etiquetado temporal.

El perfilado se adapta para leer tanto el manifiesto histórico de CWRU como el manifiesto común de NASA IMS. En este segundo caso, los snapshots ASCII sin extensión se tratan como tablas numéricas y se convierten en canales normalizados ('channel_1', 'channel_2', etc.), lo que permite obtener estadísticos ligeros sin cargar señales completas en el estado del grafo.

La limpieza se generaliza de forma acotada mediante una selección explícita de canal. El contrato 'CleaningConfig' permite indicar 'selected_channel' y el ejecutor valida esa elección contra los canales declarados en el manifiesto común. Así, NASA IMS puede limpiarse sobre un canal concreto sin habilitar transformaciones libres ni pasar todavía a estructuración o modelado.

Los agentes no interpretarán directamente carpetas crudas. Recibirán descriptores, resúmenes de manifiesto y diagnósticos de calidad generados por código determinista. De este modo pueden aumentar su capacidad de decisión –por ejemplo, recomendando canal, ventana, familia de features o modelo– sin ejecutar código arbitrario ni saltarse los contratos del sistema.

Esta elección responde a una limitación metodológica importante: los LLM locales empleados no deben considerarse fuentes autosuficientes de conocimiento sobre datasets industriales específicos. Su viabilidad se apoya en externalizar el conocimiento de dominio en expedientes verificables: métricas, candidatos, opciones ejecutables, advertencias y capacidades no soportadas. Así, el agente actúa como experto deliberativo sobre evidencia proporcionada por el sistema, no como una caja negra que recuerda parámetros de memoria.

El mismo criterio se aplica a la estructuración temporal. En lugar de pedir al LLM que invente un tamaño de ventana, el sistema calcula alternativas soportadas, estima coste, número de ventanas y riesgos de fuga de información. El estructurador puede elegir entre configuraciones válidas y justificar el compromiso entre resolución temporal, coste y robustez experimental.

Para disponer de métricas supervisadas multi-dataset sin forzar etiquetas sobre NASA IMS real, se introduce además un benchmark sintético NASA IMS-like. Este benchmark conserva formato temporal, canales y frecuencia de muestreo compatibles con NASA IMS Set 2, pero genera etiquetas 'normal' y 'fault' por construcción. Su papel metodológico es probar decisiones agénticas, persistencia y evaluación sobre un escenario no trivial; no se interpreta como resultado oficial del dataset NASA IMS.

Sobre este benchmark se incorpora además una iteración agéntica acotada. Cuando una ejecución no supera los umbrales, el sistema genera un expediente de fallo con matriz de confusión, falsos negativos, falsos positivos, umbral efectivo y convención de puntuación. El agente modelador recibe ese expediente y decide, mediante un contrato

estructurado, si propone un unico reintento o si detiene el proceso. El numero maximo de intentos queda fijado antes de la ejecucion, de modo que el agente puede aprender de sus errores sin abrir una busqueda ilimitada de hiperparametros.

La revision humana se plantea en esta fase como human-on-the-loop, no como una aprobacion manual de resultados. Al finalizar una ejecucion, el sistema puede presentar al revisor la hipotesis del agente, la evidencia usada, el cambio aplicado y el resultado observado. El humano etiqueta la calidad del razonamiento, por ejemplo como correcto, parcialmente correcto, incorrecto, inseguro o insuficiente. Esta etiqueta no modifica las metricas; se conserva como memoria supervisada para futuros prompts, RAG local o filtrado de comportamientos erroneos repetidos.

4.4. Manifiesto del dataset

El primer artefacto metodológico será un manifiesto con una fila por fichero crudo. Este manifiesto conectará los ‘.mat’ originales con el resto del pipeline. Las columnas previstas son:

Campo	Descripción
file_id	identificador del fichero original
dataset	nombre del dataset
source_path	ruta del fichero crudo
label	clase ‘normal’ o ‘fault’
fault_type	tipo de fallo si existe
fault_diameter_inch	diámetro del defecto
load_hp	carga del motor
rpm	régimen de giro
sensor_channel	canal de sensor usado
source_sample_rate_hz	frecuencia de muestreo original
target_sample_rate_hz	frecuencia objetivo del MVP
source_format	formato original
notes	observaciones

4.5. Extracción y perfilado

El canal principal inicial será ‘DE_time’, asociado al acelerómetro del lado drive-end. El perfilado generará resúmenes estadísticos por fichero y por etiqueta, incluyendo número de muestras, canales detectados, valores no finitos, media, desviación típica, RMS, curtosis, factor de cresta y energía.

4.6. Estructuración temporal

La frecuencia objetivo inicial será 12 kHz. Las ventanas candidatas serán de 2048 o 4096 muestras con solapamiento del 50 %. Esta decisión permite comparar features tabulares con modelos clásicos y conservar ventanas crudas para autoencoders posteriores.

4.7. Particiones

Para evitar fuga de información entre ventanas, no se usará un split aleatorio ingenuo. La estrategia inicial será:

- entrenamiento con ventanas normales;
- validación con ventanas normales retenidas;
- test con ventanas normales retenidas y ventanas con fallo.

4.8. Métricas

Las métricas iniciales serán precision, recall, F1-score, ROC-AUC, PR-AUC, tasa de falsos positivos y matriz de confusión. Desde el punto de vista industrial, el criterio principal será mantener alta sensibilidad ante fallos sin disparar en exceso los falsos positivos.

Capítulo 5

Implementación

5.1. Estructura del repositorio

La implementación se organiza en tres áreas:

- ‘codigo/’: aplicación, tests, datos, modelos, informes y documentación técnica;
- ‘memoria/’: documento LaTeX del TFM;
- ‘recursos/’: PDFs, referencias y diagramas auxiliares.

5.2. Estado Actual

Hasta este punto se ha creado la estructura inicial de directorios, se han descargado los datasets crudos, se ha definido el flujo de datos del MVP y se han implementado el contrato inicial del estado global del grafo y los esquemas Pydantic de entradas, decisiones y resultados. Los datos crudos se conservan en ‘codigo/data/raw/’, junto con un README de trazabilidad.

5.3. Estado global implementado

El estado global se ha dividido en dos capas:

- ‘codigo/app/schemas/state.py’: modelos Pydantic estrictos para validar el estado, configuraciones, métricas, errores y artefactos;
- ‘codigo/app/graph/state.py’: ‘TypedDict’ usado por LangGraph, validador de estado e inicializador del caso CWRU.

El inicializador ‘create_initial_cwru_state’ crea una ejecución preparada para generar el manifiesto del dataset. El estado resultante no contiene señales ni arrays, solo rutas, contexto experimental y campos reservados para artefactos futuros.

5.4. Contratos Pydantic implementados

Se han añadido tres grupos de contratos:

- ‘codigo/app/schemas/dataset.py’: manifiesto del dataset, canales y metadatos de fallo;
- ‘codigo/app/schemas/agent_decisions.py’: decisiones estructuradas del supervisor, limpiador, estructurador, modelador, evaluador y redactor;
- ‘codigo/app/schemas/executor_results.py’: resultados comunes y especializados de ejecutores deterministas.

Estos contratos consolidan la regla arquitectónica de que los agentes producen configuraciones o decisiones validables, mientras que los ejecutores devuelven rutas, métricas, errores y referencias a artefactos, pero no datos pesados.

5.5. Artefactos previstos

- ‘codigo/data/interim/cwru_bearing/manifest.csv’;
- ‘codigo/data/interim/cwru_bearing/profile.json’;
- ‘codigo/data/processed/cwru_bearing/clean_signals/’;
- ‘codigo/data/tensors/cwru_bearing/windows_features.csv’;
- ‘codigo/data/tensors/cwru_bearing/windows_raw.npz’;
- ‘codigo/data/tensors/cwru_bearing/splits.json’;
- ‘codigo/reports/cwru_bearing/’;
- ‘codigo/models/cwru_bearing/’.

5.6. Adaptadores de Entrada

Para reducir el acoplamiento al formato CWRU se ha introducido una capa ligera de adaptadores en ‘codigo/app/services/signal_adapters.py’. Esta capa expone dos operaciones comunes: leer todos los canales numéricos de un fichero y cargar un canal concreto como vector. Los ejecutores de perfilado y limpieza usan ahora esta frontera en lugar de leer directamente ficheros ‘.mat’.

La implementación inicial soporta ‘.mat’, ‘.csv’, ‘.txt’, ‘.tsv’ y ‘.npz’. En el caso de CWRU, el adaptador reconoce los canales ‘DE_time’, ‘FE_time’, ‘BA_time’ y ‘RPM’; en formatos tabulares, interpreta columnas numéricas como canales. Para incorporar un nuevo origen, como NASA IMS, el objetivo será crear o ampliar un adaptador de entrada y mantener estable el resto del pipeline: perfilado, limpieza, estructuración, modelado y evaluación.

5.7. Ejecutor de manifiesto

Se ha implementado el primer ejecutor determinista del pipeline, el módulo ‘data-set_manifest.py’. Su responsabilidad es generar el artefacto:

```
codigo/data/interim/cwru_bearing/manifest.csv
```

a partir de los ficheros ‘.mat’ crudos de CWRU.

El ejecutor no carga señales completas. Solo inspecciona los nombres de fichero y aplica un mapeo controlado de metadatos CWRU: carga, RPM, tipo de fallo, diámetro y posición del fallo exterior cuando corresponde. El manifiesto generado conserva la frecuencia original de cada fichero y fija una frecuencia objetivo de 12 kHz para el MVP.

La ejecución real sobre los datos descargados ha generado 64 registros: 4 ficheros normales y 60 ficheros con fallo. El fichero resultante contiene 65 líneas incluyendo la cabecera.

5.8. Ejecutor de perfilado

Se ha implementado el perfilador determinista ‘data_profiler.py’. Este ejecutor lee el manifiesto, inspecciona los ficheros ‘.mat’ y genera:

```
codigo/data/interim/cwru_bearing/profile.json
```

El perfil contiene metadatos y estadísticas por canal: número de muestras, valores no finitos, mínimo, máximo, media, desviación típica, RMS, curtosis y energía. No almacena arrays ni señales completas.

La ejecución real ha perfilado 64 ficheros. El resumen agregado contiene 4 ficheros normales, 60 ficheros con fallo, frecuencias de muestreo de 48 kHz y 12 kHz, y los canales detectados ‘BA_time’, ‘DE_time’, ‘FE_time’ y ‘RPM’.

5.9. Ejecutor de Limpieza

Se ha implementado el ejecutor determinista ‘cleaning.py’. Este ejecutor lee el manifiesto y el perfil, extrae el canal principal configurado, elimina valores no finitos, remuestrea la señal a 12 kHz cuando procede y guarda una señal limpia comprimida por fichero:

```
codigo/data/processed/cwru_bearing/clean_signals/
```

También genera un resumen de auditoría en:

```
codigo/data/processed/cwru_bearing/cleaning_summary.json
```

La ejecución real ha producido 64 ficheros ‘.npz’ comprimidos. Los cuatro ficheros normales se han remuestreado de 48 kHz a 12 kHz; los ficheros de fallo ya estaban a 12 kHz. El estado del grafo conserva únicamente la ruta del directorio limpio y las referencias a artefactos, no las señales.

5.10. Ejecutor de Estructuración Temporal

Se ha implementado el ejecutor determinista ‘structuring.py’. Este ejecutor lee las señales limpias, valida que el canal y la frecuencia coincidan con la configuración, genera ventanas temporales con tamaño 2048 y solapamiento del 50 %, calcula features temporales iniciales y crea particiones reproducibles a nivel de fichero.

```
codigo/data/tensors/cwru_bearing/windows_features.csv
codigo/data/tensors/cwru_bearing/windows_raw.npz
codigo/data/tensors/cwru_bearing/splits.json
```

La estrategia de partición evita mezclar ventanas de un mismo fichero entre subconjuntos. Las ventanas normales se reparten entre entrenamiento, validación y test, mientras que las ventanas con fallo se reservan para test. La ejecución real ha generado 7460 ventanas de 2048 muestras: 175 de entrenamiento, 117 de validación y 7168 de test. El tensor crudo tiene forma ‘(7460, 2048)’ y el directorio de tensores ocupa aproximadamente 22 MB.

5.11. Ejecutor de Modelado Base

Se ha implementado el ejecutor determinista ‘modeling.py’ con el primer modelo base de detección de anomalías: Isolation Forest. El modelo se entrena únicamente con ventanas normales del subconjunto de entrenamiento y consume las features temporales generadas por la fase anterior.

```
codigo/models/cwru_bearing/isolation_forest.joblib
codigo/models/cwru_bearing/predictions.csv
codigo/models/cwru_bearing/modeling_summary.json
```

El artefacto ‘joblib’ conserva el modelo entrenado, las columnas usadas y el umbral de decisión. Las predicciones guardan, por cada ventana, identificador, fichero original, partición, etiqueta real, puntuación de anomalía, umbral y predicción binaria. Las métricas finales no se calculan en este ejecutor, sino que se reservan para la fase de evaluación.

La ejecución real ha entrenado el modelo con 175 ventanas normales y ha generado 7460 predicciones. El umbral estimado ha sido 0.5992 y el resumen de modelado registra 398 ventanas predichas como normales y 7062 como anómalas. Estos conteos son una salida preliminar del modelo, no una evaluación final de rendimiento.

5.12. Ejecutor de Evaluación

Se ha implementado el ejecutor determinista ‘evaluation.py’. Este módulo lee las predicciones del modelo, compara la etiqueta binaria real con la predicción de anomalía y calcula métricas técnicas por partición. La partición principal para el MVP es ‘test’.

```
codigo/reports/cwru_bearing/evaluation/metrics.json
codigo/reports/cwru_bearing/evaluation/evaluation_summary.md
```

La ejecución real sobre el split de test ha obtenido precision 0.9991, recall 1.0000, F1-score 0.9996, ROC-AUC 1.0000, PR-AUC 1.0000 y una tasa de falsos positivos de 0.0513. La matriz de confusión ha sido: 111 verdaderos negativos, 6 falsos positivos, 0 falsos negativos y 7051 verdaderos positivos.

Estos resultados deben interpretarse como validación del pipeline y del baseline inicial sobre un benchmark controlado. No constituyen todavía una garantía de generalización industrial, que se deberá analizar con validaciones más exigentes y, posteriormente, con otros datasets como NASA IMS.

5.13. Grafo LangGraph supervisado

Se ha implementado el primer grafo LangGraph del MVP en ‘pipeline.py’ y se ha incorporado un supervisor en el modulo:

```
codigo/app/agents/supervisor.py
```

La version actual ya contempla dos modos: un supervisor LLM que emite una ‘SupervisorDecision’ validada por Pydantic y una politica determinista de respaldo para pruebas, ejecuciones locales sin modelo y recuperacion ante respuestas invalidas.

El grafo ya no avanza directamente de un ejecutor al siguiente. Todas las transiciones vuelven al supervisor:

```
START
-> supervisor
-> manifest_executor
-> supervisor
-> profiler_executor
-> supervisor
-> cleaner_agent
-> supervisor
-> cleaning_executor
-> supervisor
-> structuring_agent
-> supervisor
-> structuring_executor
-> supervisor
-> modeling_agent
-> supervisor
-> modeling_executor
-> supervisor
-> evaluator
-> supervisor
-> evaluation_agent
-> supervisor
-> report_writer
-> supervisor
-> END
```

El grafo valida el estado entrante y saliente con ‘TFMStateModel’, registra mensajes de trazabilidad, acumula referencias a artefactos y detiene la ejecucion de forma

controlada si un ejecutor devuelve estado ‘failed’. Cada decision del supervisor se almacena en ‘messages’ con rol ‘supervisor’ y contenido JSON serializado. El servicio LLM comun vive en:

```
codigo/app/services/llm.py
```

La integracion inicial usa un cliente JSON para Ollama configurable por variables de entorno. Si se activa ‘TFM_SUPERVISOR_MODE=llm’, el supervisor solicita al modelo una decision estructurada; si no hay modelo disponible o la respuesta no valida, se conserva la decision determinista como fallback.

Incluso cuando decide el LLM, la decision queda acotada por validaciones posteriores: la fase actual debe coincidir con el estado real, no se aceptan nodos fuera del plan del MVP, la revision humana permanece desactivada en esta fase local y cualquier salto invalido fuerza el fallback. De este modo se introduce toma de decisiones agentica sin permitir que el modelo ejecute codigo ni manipule datos directamente.

5.14. Agente Limpiador LLM

Se ha implementado el primer agente LLM especializado en:

```
codigo/app/agents/cleaner.py
```

El agente limpiador lee un resumen ligero del perfil del dataset y devuelve una ‘CleaningDecision’ validada por Pydantic. Su decision queda guardada como ‘cleaning_config’ en el estado, pero la transformacion real de las senales sigue delegada exclusivamente en ‘codigo/app/executors/cleaning.py’.

La fase de limpieza queda dividida en decision y ejecucion:

```
supervisor
-> cleaner_agent
-> supervisor
-> cleaning_executor
```

Con Ollama local y el modelo ‘qwen3.5:4b’, se han validado respuestas reales para el supervisor y para el limpiador. El limpiador ha propuesto ‘strategy_id = cwru_clean_llm_v1’, eliminacion de no finitos, remuestreo a 12 kHz y normalizacion ‘none’, cumpliendo el contrato ‘CleaningDecision’.

5.15. Agente Estructurador LLM

Se ha implementado el agente estructurador LLM en:

```
codigo/app/agents/structurer.py
```

Este agente decide la estructuracion temporal del dataset: tamano de ventana, solapamiento, canal principal, frecuencia objetivo, modo de etiquetas, features temporales y rutas esperadas de salida. Su salida principal es una ‘StructuringDecision’ validada por Pydantic. Igual que en el resto de fases, el agente no genera ventanas ni tensores; la ejecucion real permanece en ‘codigo/app/executors/structuring.py’.

La fase de estructuracion queda dividida en decision y ejecucion:

```
supervisor
-> structuring_agent
-> supervisor
-> structuring_executor
```

Con ‘qwen3.5:4b’, el estructurador ha generado una decision valida con ventanas de 2048 muestras, solapamiento del 50 %, canal ‘DE_time’, frecuencia de 12 kHz, modo ‘binary_anomaly’ y features temporales trazables como RMS, curtosis, crest factor y energia.

En la Fase 3 se ha ampliado ‘StructuringDecision’ con ‘comparison_candidates’. Este campo permite que el agente no solo proponga una configuracion final, sino tambien alternativas comparables con su justificacion y efecto esperado. La aplicacion no inventa esas alternativas: si el agente no las devuelve, el plan comparativo no se construye.

Esta ampliacion se materializa en el script:

```
python -m codigo.scripts.run_cwru_agentic_window_comparison
```

El script solicita a Qwen una decision de estructuracion con alternativas y, solo despues, convierte esas propuestas en un plan experimental ejecutable con el protocolo local. De este modo se conserva la separacion entre decision agentica y ejecucion reproducible, pero se evita que una regla interna suplante la deliberacion del agente.

Durante la integracion se ha detectado que ‘checkpoint_id’ es un nombre reservado por LangGraph 1.1. Por ello se ha retirado de la vista ‘TypedDict’ que circula por el grafo. La persistencia futura debera apoyarse en los checkpoints propios de LangGraph o en un campo de aplicacion que no colisione con nombres reservados.

5.16. Agente Modelador LLM

Se ha implementado el agente modelador LLM en:

```
codigo/app/agents/modeler.py
```

Este agente decide el modelo base y sus hiperparametros a partir del estado ligero del pipeline, la configuracion de estructuracion y un resumen de la tabla de features. Su salida principal es una ‘ModelingDecision’ validada por Pydantic. El agente no entrena modelos ni genera predicciones; la ejecucion real se mantiene en ‘codigo/app/executors/modeling.py’.

La fase de modelado queda dividida en decision y ejecucion:

```
supervisor
-> modeling_agent
-> supervisor
-> modeling_executor
```

Aunque el contrato ‘ModelingConfig’ contempla algoritmos futuros como One-Class SVM, LOF o autoencoders, la implementacion solo permite al agente seleccionar modelos con ejecutor reproducible. Inicialmente se incluyo ‘isolation_forest’; en Fase 3 se ha anadido PCA con error de reconstruccion como segundo modelo soportado. Tambien se acotan la semilla (‘random_state = 42’), el split de entrenamiento (‘train’),

el uso del split de validacion para el umbral y los hiperparametros permitidos, evitando que el LLM seleccione opciones que la aplicacion aun no puede ejecutar de forma reproducible.

Con ‘qwen3.5:4b’, el modelador ha generado una decision valida con ‘model_name = isolation_forest’, ‘random_state = 42’, entrenamiento sobre el split ‘train’, validacion sobre ‘validation’, ‘n_estimators = 200’, ‘n_jobs = 1’ y ‘threshold_quantile = 0.99’.

El contrato de decision del modelador incorpora ahora una lista de alternativas comparables. Este campo permite que el agente proponga otros modelos o umbrales, que despues se convierten en planes experimentales si estan soportados por el ejecutor. Con ello se aumenta la capacidad de decision del agente sin permitir modelos no implementados.

Durante esta validacion se ha ajustado el cliente de Ollama para usar ‘think=false’ por defecto en llamadas JSON estrictas, ya que este modelo puede devolver el razonamiento en el campo ‘thinking’ y dejar vacio ‘message.content’ si no se desactiva ese modo. Esta opcion no queda fijada de forma rigida: puede activarse u omitirse mediante:

```
TFM_LLM_THINK=true
TFM_LLM_THINK=auto
```

5.17. Agente Evaluador LLM

Se ha implementado el agente evaluador LLM en:

```
codigo/app/agents/evaluator.py
```

Este agente interpreta las metricas calculadas por el ejecutor determinista de evaluacion y devuelve una ‘EvaluationDecision’ validada por Pydantic. La separacion es deliberada: ‘codigo/app/executors/evaluation.py’ calcula precision, recall, F1, ROC-AUC, PR-AUC, tasa de falsos positivos y matriz de confusion; el agente solo emite el juicio tecnico, la aprobacion y las limitaciones del experimento.

La fase de evaluacion queda dividida en calculo y juicio agentico:

```
supervisor
-> evaluator
-> supervisor
-> evaluation_agent
-> supervisor
```

Para el MVP CWRU se han fijado umbrales conservadores y explicitos:

```
min_recall_required = 0.90
max_false_positive_rate = 0.10
```

El LLM puede redactar el resumen tecnico y las limitaciones, pero no puede aprobar una ejecucion que incumpla esos umbrales. Si la respuesta no valida o contradice las metricas, se activa un fallback determinista.

Tras esta decision, si ‘next_action = continue’, el supervisor ya no finaliza directamente la ejecucion: enruta la fase final hacia el agente redactor. Si el evaluador rechaza la ejecucion, el supervisor termina en ‘failed’ con el resumen de la evaluacion como motivo de parada. El fichero de metricas se genera en:

```
codigo/reports/cwru_bearing/evaluation/metrics.json
```

5.18. Agente Redactor LLM

Se ha implementado el agente redactor LLM en:

```
codigo/app/agents/report_writer.py
```

Este agente decide la estructura del informe tecnico final a partir del estado ligero del pipeline, las metricas, la evaluacion y las referencias a artefactos. Su salida principal es una ‘ReportDecision’ validada por Pydantic. El agente no escribe el informe directamente ni modifica resultados previos: la generacion material del documento se delega en el ejecutor determinista:

```
codigo/app/executors/reporting.py
```

La fase final queda dividida en decision y materializacion del informe:

```
supervisor  
-> report_writer  
-> supervisor  
-> END
```

El contrato acota la salida a formato Markdown y, en Fase 3, calcula la ruta a partir del dataset y del identificador de ejecucion para evitar sobrescrituras entre runs:

```
codigo/reports/<dataset>/<run_id>/final_report.md
```

Tambien se exigen secciones minimas: resumen ejecutivo, contexto y datos, configuraciones del pipeline, metricas y evaluacion, artefactos generados y limitaciones. Las rutas fuente propuestas por el LLM deben existir previamente en el estado como manifiesto, perfil, datos procesados, tensores, metricas o artefactos registrados. Si la respuesta no valida, se aplica un fallback determinista que genera un informe reproducible.

El resultado del ejecutor es un ‘ReportExecutorResult’ y un artefacto de tipo ‘report’. Con ello el pipeline cierra el ciclo completo del MVP: datos crudos, perfilado, limpieza, estructuracion, modelado, evaluacion, juicio agentic e informe tecnico trazable.

5.19. Persistencia Local de Ejecuciones

Como primer hito de la segunda fase se ha implementado una persistencia local de ejecuciones en:

```
codigo/app/services/run_persistence.py
```

El objetivo es conservar una evidencia ligera y auditable de cada ejecucion sin introducir todavia una base de datos externa. El servicio recibe un ‘TFMStateModel’ final y genera una carpeta por ‘run_id’ con el estado final, las decisiones extraidas de los mensajes del grafo, los artefactos, las metricas, la evaluacion, un resumen Markdown y una metadata del snapshot.

```
codigo/reports/runs/<run_id>/state_final.json
codigo/reports/runs/<run_id>/decisions.json
codigo/reports/runs/<run_id>/artifacts.json
codigo/reports/runs/<run_id>/metrics.json
codigo/reports/runs/<run_id>/evaluation.json
codigo/reports/runs/<run_id>/summary.md
codigo/reports/runs/<run_id>/snapshot.json
```

Ademas, se mantiene un indice local:

```
codigo/reports/runs/index.json
```

Esta decision mantiene separado el pipeline puro de la persistencia. La funcion ‘run_cwru_pipeline’ no se modifica; la integracion automatica se realiza mediante un wrapper de ejecucion persistida. De este modo se conserva la estabilidad del grafo ya validado y se prepara la comparacion de ejecuciones para los experimentos de la Fase 2.

5.20. Wrapper de Ejecucion Persistida

El segundo hito de la Fase 2 incorpora el wrapper:

```
run_and_persist_cwru_pipeline(...)
```

Esta funcion ejecuta primero el grafo CWRU con ‘run_cwru_pipeline’, valida el estado final y despues invoca ‘save_run_snapshot’. El resultado se devuelve como un objeto ‘PersistedPipelineRun’, compuesto por el estado final y la metadata del snapshot persistido.

La decision de implementarlo como wrapper, y no como nodo adicional del grafo, mantiene dos rutas claramente separadas: una ejecucion pura del pipeline y una ejecucion con persistencia automatica. Esto evita cambiar el comportamiento ya validado del MVP y permite que los tests sigan inyectando ejecutores y agentes simulados.

La validacion real con CWRU genera una carpeta en:

```
codigo/reports/runs/<run_id>/
```

con 13 artefactos, 17 decisiones y ningun error para una ejecucion completada. Este punto deja preparada la siguiente fase tecnica: construir un registro consultable y comparaciones entre ejecuciones persistidas.

5.21. Registro Consultable de Ejecuciones

El tercer hito de la Fase 2 introduce una capa de consulta sobre las ejecuciones persistidas:

```
codigo/app/services/run_registry.py
```

Este servicio lee el indice local de ejecuciones y los snapshots generados por la persistencia. Permite listar runs, filtrar por dataset, estado final o aprobacion, cargar la metadata de una ejecucion, recuperar sus artefactos y comparar metricas principales entre dos o mas ejecuciones.

La comparacion se devuelve mediante contratos Pydantic propios: ‘RunComparison’, ‘RunComparisonRow’ y ‘MetricComparison’. Para precision, recall y F1-score se considera mejor el valor mas alto, mientras que para la tasa de falsos positivos se considera mejor el valor mas bajo. Esta distincion permite comparar experimentos con criterio tecnico coherente con el dominio industrial.

El registro no recalcula metricas ni vuelve a ejecutar el pipeline; solo consulta evidencia ya persistida. Esta separacion prepara la futura API minima, que podra exponer consultas sobre ejecuciones sin acoplarse a LangGraph ni a los ejecutores.

5.22. Protocolo Experimental Local

El cuarto hito de la Fase 2 incorpora un servicio para ejecutar planes experimentales locales:

```
codigo/app/services/experiment_protocol.py
```

Este servicio define contratos ‘ExperimentPlan’, ‘ExperimentSpec’, ‘ExperimentRunSummary’ y ‘ExperimentPlanResult’. A partir de un plan validado, ejecuta varios runs mediante el wrapper persistido, guarda los artefactos de modelado, evaluacion e informe en carpetas separadas por experimento y genera una comparacion reutilizando ‘run_registry.py’.

El primer plan real, ‘cwru_iforest_threshold_v1’, compara el baseline de Isolation Forest con una variante que solo modifica ‘threshold_quantile’ de 0.99 a 1.00. Los resultados se guardan en:

```
codigo/experiments/cwru_local/cwru_iforest_threshold_v1/
```

La salida incluye ‘experiment_plan.json’, ‘comparison.json’ y ‘results_table.md’. Esta estructura permite llevar resultados comparables a la memoria sin recalcular metricas ni mezclar artefactos de diferentes ejecuciones.

5.23. API Minima de Consulta

El sexto hito de la Fase 2 incorpora una primera API FastAPI sobre la persistencia local:

```
codigo/app/api/app.py
codigo/app/api/routes.py
```

La API se crea mediante ‘create_app(runs_dir=...)’, de modo que en ejecucion real consulta ‘codigo/reports/runs/’ y en tests puede trabajar contra un directorio temporal. Esta primera version es deliberadamente de lectura: permite consultar runs persistidos, recuperar su metadata, listar artefactos y servir el informe Markdown asociado. Tambien expone la comparacion de ejecuciones ya persistidas, reutilizando el registro consultable sin recalcular metricas ni relanzar el pipeline.

Los endpoints iniciales son:

```
GET /health
GET /runs
GET /runs/compare
GET /runs/{run_id}
GET /runs/{run_id}/artifacts
GET /runs/{run_id}/report
```

El endpoint ‘POST /runs’ queda pospuesto hasta definir políticas de ejecución, aprobación y control de coste. Esta decisión mantiene separada la capa de consulta de la lógica que podría lanzar trabajos de modelado o experimentos múltiples.

5.24. Contratos Iniciales Multi-Dataset

Como primer paso de la Fase 3 se han incorporado contratos comunes para desacoplar el pipeline del caso CWRU sin modificar todavía los ejecutores ya validados. En ‘codigo/app/schemas/dataset.py’ se han añadido ‘DatasetDescriptor’, ‘DatasetAdapterInfo’ y ‘CommonManifestRecord’. Estos modelos permiten describir datasets heterogéneos, registrar adaptadores y representar filas de manifiesto comunes sin almacenar señales completas.

También se ha creado el servicio ‘codigo/app/services/dataset_adapters.py’, que expone un registro local de adaptadores. La primera versión incluye adaptadores para CWRU, NASA IMS y señales tabulares genéricas. Esta capa solo inspecciona rutas y cabeceras ligeras; no modifica datos crudos ni lanza transformaciones costosas.

La decisión mantiene el flujo CWRU existente como benchmark de regresión. El adaptador ‘cwru_bearing’ se ha integrado como envoltorio del manifiesto actual mediante una función común de generación de manifiestos, delegando internamente en el ejecutor CWRU ya verificado. Los tests comprueban que el CSV generado por esta ruta común coincide con el manifiesto producido por la ruta histórica.

Para NASA IMS, el adaptador reconoce rutas o ficheros con patrón temporal compatible, infiere canales desde ficheros sintéticos pequeños y registra advertencias cuando detecta recuentos de canales inconsistentes. Tras la inspección local, se ha implementado ‘build_manifest(...)’ solo para carpetas preextraídas. Esta función genera un manifiesto común con una fila por snapshot, ‘label = unknown’, fallo final conocido a nivel de set, frecuencia de 20 kHz, canales serializados como JSON y metadatos escalares sobre puntos por fichero, duración nominal, carga, régimen y variante local.

La inspección local ha confirmado que el paquete disponible se organiza como ‘zip ->7z ->rar’, con tres archivos RAR de experimentos y un README interno. El README describe tres ensayos test-to-failure con snapshots ASCII de un segundo, 20.480 puntos por fichero y frecuencia de muestreo de 20 kHz. También se ha detectado una discrepancia local en el tercer RAR, que lista más ficheros y una ruta interna distinta a la descrita por el README. Por ello, la primera implementación de manifiesto NASA IMS trabaja con carpetas preextraídas y sintéticas antes de procesar el paquete completo. La ingestión directa del contenedor anidado queda pospuesta hasta decidir dependencias y experiencia de interfaz.

El perfilador ‘data_profiler.py’ se ha generalizado para detectar si el CSV de entrada sigue el formato histórico CWRU o el contrato común. Para NASA IMS, propaga ‘record_id’, ‘run_id’, ‘condition_id’, ‘primary_channel’, canales de manifiesto y metadatos JSON. La capa ‘signal_adapters.py’ también reconoce ficheros IMS con nombre

temporal y sin extension, expone ‘read_signal_frame’ para leerlos como tablas numericas con ‘pandas’ y asigna canales normalizados ‘channel_1’, ‘channel_2’, etc. Esta adaptacion permite generar perfiles estadisticos sobre estructuras NASA preextraidas.

El perfil incluye ademas un bloque ‘decision_summary’ orientado al agente limpiador. Este bloque clasifica si el perfil esta limpio, requiere remuestreo, requiere seleccionar canal o presenta calidad insuficiente. Tambien enumera acciones requeridas, canales candidatos, opciones de limpieza soportadas y advertencias bloqueantes o no bloqueantes. De este modo, el LLM local no necesita conocer de memoria el dataset industrial: recibe un expediente experto calculado por el sistema y decide dentro de un espacio de accion validado.

El ejecutor ‘cleaning.py’ tambien se ha generalizado para consumir manifiestos comunes. El contrato ‘CleaningConfig’ incorpora ‘selected_channel’; si se declara, el ejecutor carga ese canal mediante la capa de adaptadores de senal y lo valida contra ‘channel_names’ en los manifiestos comunes. Si no se declara, mantiene compatibilidad con ‘sensor_channel’ para CWRU y con ‘primary_channel’ para NASA IMS. La salida ‘.npz’ conserva metadatos ligeros como ‘record_id’, ‘dataset’, ‘run_id’, ‘condition_id’ y ‘label_detail’, manteniendo la estructura comun necesaria para estructuracion posterior.

El ejecutor ‘structuring.py’ incorpora un resumen experto para el agente estructurador. A partir de las señales limpias estima configuraciones soportadas de ventana y solape, numero aproximado de ventanas, coste relativo, conjuntos de features temporales disponibles y advertencias de fuga de informacion por fichero, run o secuencia temporal. El agente puede elegir entre estas configuraciones, pero la validacion mantiene los valores dentro de rangos controlados y las features dentro de las implementadas por el ejecutor.

5.25. Prueba Agentica NASA IMS con Qwen

Para validar el comportamiento de los agentes locales ante un dataset distinto de CWRU se ha añadido un script de prueba controlada:

```
python -m codigo.scripts.run_nasa_ims_qwen_smoke \
  --model qwen3.5:4b \
  --run-id nasa-ims-qwen-smoke-fase3-qwen
```

La prueba crea una carpeta NASA IMS sintetica preextraida, ejecuta el grafo completo con agentes Qwen/Ollama y persiste el snapshot en:

```
codigo/reports/runs/nasa-ims-qwen-smoke-fase3-qwen/
```

El objetivo de esta ejecucion no es entrenar un modelo NASA IMS, sino comprobar si los agentes usan los expedientes expertos. El limpiador selecciono ‘channel_1’ como canal principal, el estructurador eligio ventanas de 1024 muestras con solape del 50 % y features temporales soportadas, y el modelador propuso ‘isolation_forest’, coherente con el unico ejecutor de modelado disponible en ese momento.

La ejecucion genero manifiesto, perfil, señales limpias, features, tensores y splits. Sin embargo, el flujo termino en estado ‘failed’ por una barrera metodologica deliberada: NASA IMS no dispone todavia de etiquetas por ventana ni de una politica de particion temporal defendible. Esta prueba confirma que los agentes locales pueden

tomar decisiones utiles con contexto experto, pero tambien que debe formalizarse una decision de capacidad no soportada antes de permitir el paso al modelador en datasets run-to-failure.

5.26. Benchmark NASA IMS Sintetico Etiquetado

Para obtener metricas supervisadas sin atribuir etiquetas por ventana al NASA IMS real, se ha añadido un generador sintetico controlado:

```
codigo/app/services/synthetic_nasa_ims.py
codigo/scripts/run_nasa_ims_synthetic_temporal_benchmark.py
```

El generador crea una estructura compatible con NASA IMS Set 2: ficheros con nombre temporal, cuatro canales tabulares, frecuencia de 20 kHz y un manifiesto comun con etiquetas ‘normal’ y ‘fault’ generadas por construccion. Esta distincion es importante: el benchmark valida el flujo multi-dataset y las decisiones agenticas, pero no representa anotaciones oficiales del dataset NASA IMS.

La ejecucion canonica con Qwen se lanza mediante:

```
python -m codigo.scripts.run_nasa_ims_synthetic_temporal_benchmark \
--model qwen3.5:4b \
--run-id nasa-ims-synth-agentic-qwen-fase3
```

La run deja snapshot persistido en ‘codigo/reports/runs/nasa-ims-synth-agentic-qwen-fase3/’. Qwen selecciono ‘channel_1’, elige ventanas de 2048 muestras con 50 % de solape, propuso Isolation Forest y, finalmente, el agente evaluador rechazo la ejecucion por recall insuficiente y FPR superior al umbral local. Esta salida es metodologicamente valiosa porque introduce un caso no perfecto y obliga a una iteracion posterior de decisiones, en lugar de cerrar todos los experimentos con recall 1.0000.

5.27. Bucle de Reintento Agentico

Para convertir una ejecucion rechazada en aprendizaje trazable se ha añadido un servicio de analisis de fallo y un contrato especifico de reintento:

```
codigo/app/services/iteration_analysis.py
codigo/scripts/run_nasa_ims_agentic_retry.py
ModelingRetryDecision
```

El servicio ‘iteration_analysis.py’ resume la ejecucion fallida sin exponer el dataset completo al LLM: calcula matriz de confusion, falsos negativos, falsos positivos, estadisticos de puntuacion, umbral efectivo y reintentos restantes. El agente modelador recibe ese expediente y debe devolver JSON validable con ‘should_retry’, ‘retry_config’, ‘learning_summary’, ‘expected_effect’ y ‘stop_reason’ cuando decida parar.

La logica de ejecucion sigue siendo determinista. El agente no entrena modelos ni modifica predicciones; solo propone una nueva ‘ModelingConfig’ soportada. El script ejecuta como maximo dos intentos, persiste cada decision y, si el ultimo intento sigue rechazado, fuerza ‘next_action = stop’ en la evaluacion efectiva para impedir bucles infinitos. Esta parada no sustituye al juicio del agente: documenta el limite operativo pactado para la ejecucion.

Para auditar si el agente razona de forma consistente, se ha añadido una capa de post-mortem:

```
codigo/app/services/reasoning_audit.py
codigo/app/schemas/reasoning.py
codigo/scripts/create_agentic_reasoning_audit.py
```

El post-mortem compara hipotesis, evidencia, accion, efecto esperado y metricas antes/despues. Ademas puede generar una solicitud de revision humana y una plantilla JSON de veredicto. Esta revision humana no altera la ejecucion; sirve para decidir si el razonamiento debe reutilizarse como contexto, descartarse o tratarse como caso frontera.

Como pulido adicional de trazabilidad multi-dataset, los artefactos comunes ya no usan nombres internos heredados de CWRU cuando se ejecutan sobre NASA IMS. Las rutas pueden seguir apuntando a 'cwru_bearing' en el benchmark historico, pero los identificadores de artefacto pasan a ser neutrales: 'windows_features', 'model_predictions', 'modeling_summary', 'evaluation_metrics' y 'evaluation_summary'. Esto reduce el acoplamiento visual del sistema a CWRU y hace mas clara la lectura de snapshots multi-dataset.

5.28. Dependencias

Se ha creado 'requirements.txt' como punto inicial para construir el entorno de Miniconda 'tfm_v2', fijado sobre Python 3.11. Incluye dependencias de contratos y grafo, procesamiento de datos, modelos clásicos, API local con FastAPI, visualización técnica y verificación.

5.29. Verificación Actual

La verificación de los contratos se ha realizado con tests unitarios en el entorno objetivo:

```
conda run -n tfm_v2 python -m unittest discover codigo/tests
```

Las pruebas validan la creación del estado inicial CWRU, serialización JSON, rechazo de campos no declarados, restricciones de configuraciones y métricas, coherencia del manifiesto, decisiones de agentes, resultados de ejecutores y generación del manifiesto, perfilado, limpieza, estructuración temporal y modelado base y evaluación CWRU. También validan los adaptadores de entrada para formatos '.mat' y '.csv', y la orquestación del grafo minimo con ejecutores simulados. Tambien validan el supervisor LLM con cliente fake, el agente limpiador LLM, el agente estructurador LLM, el agente modelador LLM, el agente evaluador LLM, el agente redactor LLM, el ejecutor de informes, el parser JSON, la persistencia local de ejecuciones, el registro consultable, el protocolo experimental local, la API minima de consulta, la configuracion de razonamiento de Ollama, los contratos multi-dataset iniciales, el registro de adaptadores de dataset, el manifiesto comun NASA IMS sobre una carpeta sintetica preextraida, el rechazo de paquetes comprimidos para ese manifiesto, el perfilado del manifiesto comun NASA IMS, la lectura tabular de snapshots IMS sin extension, la limpieza comun con seleccion controlada de canal, el rechazo de canales no declarados, el rechazo de transiciones invalidas y el fallback determinista contextualizado por dataset. Tambien validan el 'decision_summary' del perfilador, su entrega al agente limpiador, el expediente experto del estructurador, las alternativas comparables propuestas por el estructurador y

la generacion de planes de ventana y modelo a partir de dichas alternativas, el ejecutor PCA, el diagnostico no supervisado de degradacion, el benchmark NASA sintetico, la iteracion agentica acotada, la auditoria del razonamiento y la preparacion de revision humana. En esta fase se han ejecutado 168 tests correctamente.

5.30. Siguiente Paso de Implementación

Tras incorporar la consulta y comparacion de runs persistidos por API, la siguiente fase no se plantea como una simple ampliacion de endpoints, sino como una validacion mas exigente del nucleo multiagente. La prioridad sera extender el sistema a datasets industriales mas complejos, incorporar nuevos modelos con ejecutores deterministas y ampliar el espacio de decision de los agentes bajo contratos estrictos.

El endpoint ‘POST /runs‘ y el punto Human Review quedan ligados a esa evolucion: solo deberian implementarse cuando existan politicas claras de validacion de solicitudes, control de coste, limites de recursos y aprobacion humana para ejecuciones potencialmente costosas.

Capítulo 6

Experimentos

6.1. Protocolo experimental

La Fase 2 introduce un protocolo experimental local para pasar de una unica ejecucion valida a un conjunto pequeno de ejecuciones comparables. El protocolo se implementa en:

```
codigo/app/services/experiment_protocol.py
```

La unidad principal es un manifiesto ‘ExperimentPlan’, compuesto por varias entradas ‘ExperimentSpec’. Cada entrada fija un ‘run_id’, una descripcion y una configuracion de modelado validada mediante ‘ModelingConfig’. La ejecucion del plan reutiliza el wrapper persistido del pipeline y, al finalizar, invoca el registro consultable para comparar metricas entre runs.

Cada plan conserva tres artefactos agregados:

- ‘experiment_plan.json’, con el manifiesto experimental;
- ‘comparison.json’, con la comparacion estructurada;
- ‘results_table.md’, con la tabla resumida para documentacion.

Ademas, los modelos, predicciones, metricas e informes finales se escriben dentro de la carpeta de cada experimento para evitar sobrescrituras entre runs comparables.

6.2. Plan CWRU Isolation Forest Threshold

El primer plan experimental real se identifica como:

```
cwru_iforest_threshold_v1
```

El objetivo es estudiar una variacion controlada del umbral de decision del modelo base Isolation Forest. No se introduce ningun algoritmo nuevo, ya que la regla de la arquitectura exige que todo modelo disponible para un agente tenga un ejecutor determinista reproducible.

Los dos runs ejecutados han sido:

- ‘cwru_iforest_threshold_v1_baseline_threshold_099’;

- ‘cwru_iforest_threshold_v1_conservative_threshold_100’.

Ambos mantienen constante el dataset, la limpieza, la estructuracion temporal, el algoritmo, la semilla y ‘n_estimators=200’. La unica variable modificada es ‘threshold_quantile’: 0.99 para el baseline y 1.00 para la variante conservadora.

Tabla 6.1: Comparacion local de umbral con Isolation Forest sobre CWRU

Experimento	Umbral	Precision	Recall	F1	FPR
Baseline	0.99	0.9991	1.0000	0.9996	0.0513
Conservador	1.00	0.9996	1.0000	0.9998	0.0256

Los dos runs quedan aprobados por el agente evaluador. La variante conservadora reduce la tasa de falsos positivos de 0.0513 a 0.0256 y mejora ligeramente la precision y el F1-score, manteniendo recall 1.0000. En este benchmark, por tanto, el umbral conservador resulta preferible para reducir falsas alarmas sin penalizar la deteccion de fallos.

Los artefactos agregados del plan quedan en:

`codigo/experiments/cwru_local/cwru_iforest_threshold_v1/`

6.3. Comparacion Agentica de Ventanas CWRU

La Fase 3 incorpora un experimento donde el agente estructurador no solo elige una configuracion, sino que propone alternativas comparables. La ejecucion se lanzo con:

```
python -m codigo.scripts.run_cwru_agentic_window_comparison \
  --model qwen3.5:4b \
  --plan-id cwru-agentic-window-qwen-fase3
```

Qwen propuso tres configuraciones de ventana, todas con solape del 50 %: 2048 muestras como seleccion principal, 1024 muestras para aumentar resolucion temporal y 4096 muestras para aumentar contexto por ventana. El protocolo experimental materializo esas propuestas sin anadir configuraciones no decididas por el agente.

Tabla 6.2: Comparacion agentica de ventanas sobre CWRU

Ventana	Solape	Precision	Recall	F1	FPR
2048	0.50	0.9991	1.0000	0.9996	0.0513
1024	0.50	0.9996	0.9999	0.9998	0.0212
4096	0.50	0.9991	1.0000	0.9996	0.0517

La configuracion de 1024 muestras obtuvo la mejor precision, el mejor F1-score y la menor tasa de falsos positivos. La configuracion principal de 2048 muestras mantuvo recall 1.0000, pero produjo mas falsas alarmas. Este resultado apoya la idea central del sistema: los agentes formulan hipotesis tecnicas y la aplicacion las convierte en evidencia experimental trazable.

Los artefactos quedan en:

`codigo/experiments/cwru_local/cwru-agentic-window-qwen-fase3/`

6.4. Comparacion Agentica de Modelos CWRU

Tras seleccionar la ventana de 1024 muestras, se ejecuto una comparacion agentica de modelos. El modelador propuso Isolation Forest, PCA con error de reconstruccion y una variante conservadora de Isolation Forest:

```
python -m codigo.scripts.run_cwru_agentic_model_comparison \
--model qwen3.5:4b \
--plan-id cwru-agentic-model-qwen-fase3
```

Tabla 6.3: Comparacion agentica de modelos sobre CWRU

Modelo	Umbral	Precision	Recall	F1	FPR
Isolation Forest	0.99	0.9996	0.9999	0.9998	0.0212
PCA reconstruction	0.99	1.0000	0.9623	0.9808	0.0000
Isolation Forest cons.	1.00	0.9999	0.9994	0.9996	0.0085

PCA elimina falsos positivos en este protocolo, pero reduce el recall a 0.9623. Esta comparacion es relevante porque rompe el patron de resultados perfectos y muestra un trade-off mas creible entre sensibilidad y falsas alarmas.

6.5. Prueba Smoke NASA IMS con Agentes Qwen

Dentro de la Fase 3 se ha ejecutado una prueba smoke sobre una estructura NASA IMS sintetica preextraida. La ejecucion usa el grafo completo y agentes Qwen/Ollama, pero bloquea deliberadamente el modelado final porque el dataset run-to-failure aun no dispone de etiquetas por ventana ni de una politica de particion temporal validada.

```
python -m codigo.scripts.run_nasa_ims_qwen_smoke \
--model qwen3.5:4b \
--run-id nasa-ims-qwen-smoke-fase3-qwen
```

El snapshot queda persistido en:

```
codigo/reports/runs/nasa-ims-qwen-smoke-fase3-qwen/
```

La prueba registro 12 decisiones agenticas. Entre ellas, el limpiador selecciono ‘channel_1’, el estructurador eligio ventanas de 1024 muestras con solape del 50% y el modelador propuso ‘isolation_forest’. El resultado final fue ‘failed’, no por fallo de gestion o procesamiento, sino por guardarrail metodologico. Por tanto, esta ejecucion se interpreta como validacion de la adaptacion inicial de agentes y artefactos, no como experimento de rendimiento NASA IMS.

Para evitar presentar metricas supervisadas sin etiquetas defendibles, se ha anadido un diagnostico no supervisado de degradacion sobre las features NASA generadas:

```
python -m codigo.scripts.run_nasa_ims_degradation_diagnostics
```

El diagnostico no calcula recall ni F1. Sobre la prueba sintetica NASA IMS detecta una tendencia creciente: media inicial 0.3080, media final 0.6866, ratio final/inicial 2.2296 y pendiente temporal 0.1893.

6.6. Benchmark NASA IMS Sintetico Etiquetado

Para obtener una evaluacion supervisada multi-dataset sin usar etiquetas inventadas sobre NASA IMS real, se ejecuto un benchmark sintetico temporal. La estructura mantiene el formato NASA IMS Set 2, pero las etiquetas binarias son controladas por construccion.

```
python -m codigo.scripts.run_nasa_ims_synthetic_temporal_benchmark \
--model qwen3.5:4b \
--run-id nasa-ims-synth-agentic-qwen-fase3
```

Tabla 6.4: Run agentica sobre benchmark NASA IMS sintetico etiquetado

Modelo	Precision	Recall	F1	ROC-AUC	FPR
Isolation Forest	0.9130	0.6000	0.7241	0.8041	0.1429

La ejecucion quedo rechazada por el agente evaluador, ya que no cumple el recall minimo de 0.90 ni la tasa maxima de falsos positivos de 0.10. Este resultado es deliberadamente util: aporta un caso no perfecto sobre un dataset distinto de CWRU y muestra que el sistema no solo informa exitos, sino tambien fallos tecnicos que requieren una nueva iteracion de configuracion.

6.7. Iteracion Agentica sobre NASA IMS Sintetico

A partir de la run rechazada se ejecuto un bucle de reintento acotado, con dos intentos maximos. El modelador recibio un analisis de fallo con falsos negativos, falsos positivos y convencion del umbral. En ambos intentos Qwen interpreto que el problema principal era la perdida de anomalias y propuso bajar ‘threshold_quantile’.

Tabla 6.5: Reintentos agenticos sobre benchmark NASA IMS sintetico

Run	Umbral	Precision	Recall	F1	FPR
Base	0.99	0.9130	0.6000	0.7241	0.1429
Retry 1	0.95	0.8846	0.6571	0.7541	0.2143
Retry 2	0.50	0.7143	1.0000	0.8333	1.0000

El resultado ilustra el comportamiento buscado para el TFM. El agente aprende de los falsos negativos y actua en la direccion tecnica correcta: reducir el umbral aumenta el recall. Sin embargo, el segundo reintento muestra tambien el riesgo industrial de optimizar solo sensibilidad: se alcanza recall 1.0000, pero todos los normales del split de test pasan a clasificarse como anomalias. La ejecucion queda rechazada por FPR excesiva y el sistema detiene el bucle con ‘next_action = stop’ al agotarse el presupuesto de reintentos. Por tanto, la arquitectura permite deliberacion y correccion agentica, pero conserva un limite explicito contra busquedas indefinidas o aprobaciones engañosas.

Sobre estos reintentos se genero una auditoria de razonamiento. El primer caso se clasifico automaticamente como ‘partially_supported’, ya que la hipotesis mejoro recall y F1 pero mantuvo FPR por encima del umbral. El segundo se clasifico como ‘overcorrected’: la direccion del razonamiento era correcta, pero la magnitud del cambio

fue insegura. En ambos casos se creo una solicitud de revision humana para que el razonamiento pueda etiquetarse posteriormente como ejemplo positivo, negativo o caso frontera antes de incorporarlo a una memoria contextual.

6.8. Experimentos posteriores

Experimentos previstos:

- comparación entre features temporales y frecuenciales;
- validación leave-one-load-out;
- refinamiento de la iteracion agentica para que el modelador compare tambien cambios de modelo, no solo de umbral;
- validación sobre NASA IMS real cuando exista una politica temporal de etiquetado defendible.

Capítulo 7

Resultados

7.1. Resultados cuantitativos

La validacion cuantitativa se ha organizado en dos niveles. Primero se conserva el benchmark CWRU como referencia de regresion del MVP. Despues se incorporan experimentos de Fase 3 donde los agentes locales proponen alternativas de ventana, modelo y reintento sobre evidencia persistida.

7.1.1. Baseline CWRU de Fase 2

La Fase 2 incorpora una comparacion local entre dos ejecuciones persistidas del baseline Isolation Forest sobre CWRU. Ambas ejecuciones mantienen constante el dataset, la limpieza, la estructuracion temporal, la semilla y ‘n_estimators=200’; la unica variable modificada es el cuantil usado para fijar el umbral de anomalia.

Experimento	Umbral	Precision	Recall	F1	FPR
Baseline	0.99	0.9991	1.0000	0.9996	0.0513
Conservador	1.00	0.9996	1.0000	0.9998	0.0256

Tabla 7.1: Comparacion de umbral con Isolation Forest en el split de test CWRU.

La variante conservadora reduce los falsos positivos sin penalizar el recall. En el baseline, la matriz de confusion del split de test contiene 111 verdaderos negativos, 6 falsos positivos, 0 falsos negativos y 7051 verdaderos positivos. En la variante con umbral 1.00, los falsos positivos bajan a 3, con 114 verdaderos negativos, 0 falsos negativos y 7051 verdaderos positivos.

7.1.2. Comparacion agentica de ventanas

En Fase 3 el agente estructurador Qwen no recibe una unica configuracion fija, sino que puede proponer alternativas comparables. Sobre CWRU propuso tres ventanas: 2048 muestras como opcion principal, 1024 muestras para mayor resolucion temporal y 4096 muestras para mayor contexto por ventana. El protocolo experimental materializo solo las alternativas propuestas por el agente y persistio cada run.

La alternativa de 1024 muestras obtuvo el mejor F1 y la menor tasa de falsos positivos. Este resultado es relevante porque la mejora no procede de una busqueda

Ventana	Precision	Recall	F1	FPR
1024	0.9996	0.9999	0.9998	0.0212
2048	0.9991	1.0000	0.9996	0.0513
4096	0.9991	1.0000	0.9996	0.0517

Tabla 7.2: Comparacion agentica de tamanos de ventana en CWRU con 50 % de solape.

exhaustiva codificada, sino de una hipotesis propuesta por el agente y despues evaluada por ejecutores deterministas.

7.1.3. Comparacion agentica de modelos

La Fase 3 tambien amplio el espacio de decision del modelador con PCA basado en error de reconstruccion. Qwen propuso comparar Isolation Forest, PCA y una variante conservadora de Isolation Forest usando la ventana de 1024 muestras seleccionada en el experimento anterior.

Modelo	Umbral	Precision	Recall	F1	FPR
Isolation Forest	0.99	0.9996	0.9999	0.9998	0.0212
PCA reconstruction	0.99	1.0000	0.9623	0.9808	0.0000
Isolation Forest conservador	1.00	0.9999	0.9994	0.9996	0.0085

Tabla 7.3: Comparacion agentica de modelos sobre CWRU con ventana de 1024 muestras.

PCA ofrece un intercambio distinto: elimina falsos positivos en este protocolo, pero reduce el recall. Este resultado evita que toda la evidencia de la fase dependa de metricas perfectas y muestra que el agente puede comparar alternativas con ventajas y limitaciones tecnicas diferentes.

7.1.4. Benchmark NASA sintetico y reintentos

NASA IMS real se mantiene bloqueado para metricas supervisadas hasta definir una politica temporal y de etiquetado defendible. Para obtener un caso no trivial sin atribuir etiquetas artificiales al dataset real, se genero un benchmark sintetico con forma de entrada NASA IMS y etiquetas conocidas por construccion.

La primera ejecucion agentica obtuvo precision 0.9130, recall 0.6000, F1 0.7241, ROC-AUC 0.8041, PR-AUC 0.8842 y FPR 0.1429. El evaluador rechazo la run al detectar que las metricas no cumplian los umbrales locales. Despues se activo un bucle de reintento acotado donde el modelador recibia analisis de error y decidia si modificar la configuracion o detenerse.

El primer reintento mejora recall y F1, aunque aumenta la tasa de falsos positivos. El segundo reintento recupera todas las anomalias, pero clasifica tambien todos los normales de test como anomalos. El evaluador rechaza esta sobrecorreccion y el bucle se detiene por limite de intentos. El resultado es valioso porque muestra aprendizaje local de errores, pero tambien control ante una mejora aparente que seria inaceptable operativamente.

Run	Umbral	Precision	Recall	F1	FPR
Base	0.99	0.9130	0.6000	0.7241	0.1429
Reintento 1	0.95	0.8846	0.6571	0.7541	0.2143
Reintento 2	0.50	0.7143	1.0000	0.8333	1.0000

Tabla 7.4: Reintentos agenticos sobre benchmark NASA sintetico etiquetado.

7.2. Resultados cualitativos

La prueba smoke con agentes Qwen sobre una carpeta NASA IMS sintetica preextraida confirma que el patron de expediente experto y decision estructurada es viable con LLM locales. El limpiador selecciono un canal a partir del perfil, el estructurador eligio una configuracion temporal soportada y el modelador propuso un modelo ejecutable. Sin embargo, la run termino en estado ‘failed‘ por guardarrail metodologico: NASA IMS no debe evaluarse como problema supervisado hasta definir etiquetas por ventana y particion temporal sin fuga.

La Fase 3 tambien introduce auditoria del razonamiento. Cada reintento puede generar un post-mortem que compara la hipotesis declarada por el agente con el resultado real. En el benchmark NASA sintetico, el primer reintento queda como ‘partially_supported‘ y el segundo como ‘overcorrected‘. Con el trigger de revision humana se genera ademas una plantilla para que una persona etiquete si el razonamiento debe reutilizarse, descartarse o conservarse como caso frontera.

Este punto refuerza la aportacion central del TFM: el sistema no solo registra metricas finales, sino tambien por que un agente cambio una configuracion, que efecto esperaba y si la evidencia posterior respalda ese razonamiento.

7.3. Limitaciones observadas

Los resultados sobre CWRU deben interpretarse como validacion del pipeline y del protocolo experimental, no como garantia de generalizacion industrial. CWRU es un benchmark controlado, con condiciones experimentales limpias y diferencias claras entre normalidad y fallo.

NASA IMS real aporta mayor interes industrial, pero no se ha usado para metricas supervisadas porque carece de etiquetas por ventana y requiere una politica temporal defendible. La version sintetica permite estudiar el comportamiento multiagente ante metricas no perfectas, pero no debe presentarse como resultado oficial sobre NASA IMS real.

Otra limitacion es que los agentes locales siguen dependiendo de la calidad del expediente experto que reciben. El sistema evita delegar conocimiento de nicho en la memoria parametrica del LLM, pero debe seguir ampliando diagnosticos, modelos soportados y revisiones humanas para que la autonomia agentic sea defendible en escenarios mas variados.

Capítulo 8

Conclusiones

8.1. Conclusiones principales

El trabajo desarrollado demuestra que una arquitectura multiagente puede ser viable para procesamiento de datos industriales y detección de anomalías si se separa con claridad la decisión generativa de la ejecución determinista. Los agentes proponen configuraciones, comparan alternativas y justifican cambios; los ejecutores validan contratos, aplican transformaciones reproducibles y persisten artefactos, métricas y limitaciones.

La primera aportación es una arquitectura local trazable basada en LangGraph, contratos Pydantic y persistencia por run. Esta base permite reproducir el flujo completo sobre CWRU, consultar ejecuciones anteriores y comparar métricas sin mezclar razonamiento del agente con manipulación arbitraria de datos.

La segunda aportación es la generalización inicial a datasets heterogéneos. NASA IMS se ha incorporado mediante contratos comunes, adaptadores de manifiesto, perfilado enriquecido, selección controlada de canal y estructuración temporal. La evaluación supervisada sobre NASA IMS real se ha bloqueado de forma deliberada hasta definir etiquetas temporales y particiones sin fuga, lo que refuerza la validez metodológica del sistema.

La tercera aportación es el aumento de protagonismo agéntico sin eliminar guardarrailes. Los agentes Qwen locales han usado expedientes expertos para elegir canales, ventanas, modelos y reintentos. También han generado alternativas comparables y razonamientos que después se han auditado frente a los resultados reales. El sistema muestra que un LLM local no necesita conocer de memoria datasets industriales de nicho si recibe evidencia estructurada, opciones soportadas y límites claros.

La cuarta aportación es experimental. La comparación agéntica de ventanas y modelos sobre CWRU muestra mejoras y trade-offs medibles. En particular, PCA con error de reconstrucción reduce falsos positivos a costa de recall, mientras que Isolation Forest mantiene mayor sensibilidad. El benchmark NASA sintético introduce un caso no perfecto donde el agente aprende de falsos negativos, prueba una hipótesis y se detiene cuando la mejora de recall provoca una tasa de falsos positivos inaceptable.

8.2. Limitaciones

Las métricas de CWRU no deben interpretarse como garantía de rendimiento en planta industrial. CWRU es un benchmark controlado y muy separable. NASA IMS es

mas representativo de degradacion temporal, pero en esta fase solo se ha usado para ingestion, perfilado, diagnostico y pruebas sinteticas etiquetadas. La validacion supervisada sobre NASA IMS real queda pendiente de una politica de etiquetado temporal defendible.

El sistema tambien depende de la calidad de los expedientes expertos que reciben los agentes. Esta decision es intencionada: se evita pedir a un LLM local que invente conocimiento de dominio, pero obliga a mantener diagnosticos, contratos, registros de modelos y revisiones humanas bien documentados.

8.3. Trabajo futuro

Las lineas principales son:

- definir una politica temporal defendible para NASA IMS real y otros datasets run-to-failure;
- incorporar modelos adicionales con ejecutores deterministas, como One-Class SVM, LOF y autoencoders, solo cuando existan tests, persistencia y evaluacion comparable;
- ampliar la experimentacion multi-dataset con planes acotados y comparacion frente a politicas deterministas;
- definir ejecuciones controladas desde API y Human Review antes de permitir trabajos costosos mediante ‘POST /runs’;
- construir en una fase posterior una memoria agentica supervisada, basada en post-mortems y revisiones humanas, que pueda alimentar un RAG local sin modificar metricas ni saltarse contratos;
- preparar Docker, SLURM y checkpoints persistentes cuando el flujo local y la politica de ejecucion esten suficientemente estabilizados.

Bibliografía

- [1] Case Western Reserve University Bearing Data Center. 12k drive end bearing fault data, 2026. Disponible en: <https://engineering.case.edu/bearingdatacenter/12k-drive-end-bearing-fault-data>.
- [2] Case Western Reserve University Bearing Data Center. Bearing data center, 2026. Disponible en: <https://engineering.case.edu/bearingdatacenter/welcome>.
- [3] Case Western Reserve University Bearing Data Center. Normal baseline data, 2026. Disponible en: <https://engineering.case.edu/bearingdatacenter/normal-baseline-data>.
- [4] NASA Prognostics Center of Excellence. Data set repository, 2026. Disponible en: <https://www.nasa.gov/intelligent-systems-division/discovery-and-systems-health/pcoe/pcoe-data-set-repository/>.
- [5] NASA Prognostics Center of Excellence. Ims bearings dataset, 2026. Disponible en: <https://phm-datasets.s3.amazonaws.com/NASA/4.+Bearings.zip>.